

Instituto Tecnológico de Las Américas (ITLA)
Programación III

PROYECTO FINAL

Sistema de Reservas de Citas

Nombre: Brian Amaury Guzman Mañon

Matrícula: 20241286

Profesor: Kelyn Tejada

Índice

Índice.....	2
Introducción	3
1. Estrategia de Trabajo (Planificación)	3
1.1 Nombre del proyecto de software	3
1.2 Tecnología para aplicar	3
1.3 Objetivo del proyecto.....	3
1.4 Alcance del proyecto	4
1.5 Cronograma del proyecto.....	4
1.6 Definición del primer Release.....	4
1.6.1 Requerimientos funcionales.....	5
1.6.2 Requerimientos no funcionales.....	5
2. Metodología Scrum	6
2.1 Definición de tareas a ejecutar.....	6
2.2 Definición del equipo de trabajo	6
2.3 Herramientas y definición de épicas	6
2.4 Ceremonias de Scrum	6
2.5 Historias de usuario.....	7
3. Plan de Pruebas.....	8
3.1 Requerimientos relacionados con las historias de usuario.....	8
3.2 Criterios de aceptación y rechazo	8
3.3 Herramientas de pruebas justificadas	8
3.4 Cronograma de ejecución de pruebas.....	8
3.5 Plantillas para casos de prueba y equipos de pruebas	9
3.6 Plan de automatización de pruebas	10
3.7 Ejecución y demostración	10
4. Demostración y Entregables.....	12
4.1 Video	12
4.2 Links funcionales	12
Conclusiones	13
Bibliografía	14

Introducción

Este documento presenta el proyecto final de la asignatura Programación III: un sistema de reservas de citas construido aplicando la metodología Agile-Scrum de principio a fin. El documento cubre la planificación del proyecto, la aplicación práctica de Scrum (equipo, épicas, historias de usuario), el plan de pruebas y la evidencia de su ejecución, y los entregables de la demostración.

El sistema resuelve un problema real y acotado: automatizar el agendamiento de citas entre clientes y profesionales, aplicando un conjunto de reglas de negocio que evitan los errores más comunes de una agenda manual (citas duplicadas, profesionales comprometidos en dos horarios a la vez, reservas fuera de horario laboral). Cada una de esas reglas fue construida siguiendo un ciclo de prueba-antes-que-código, lo cual se documenta en la sección 3 junto con la evidencia real de su ejecución.

1. Estrategia de Trabajo (Planificación)

1.1 Nombre del proyecto de software

Sistema de Reservas de Citas: una API REST para la gestión de clientes, profesionales y el agendamiento de citas entre ellos, con validación automática de las reglas de disponibilidad del negocio.

1.2 Tecnología para aplicar

Las siguientes herramientas y lenguajes se emplearon en el desarrollo del sistema, junto con la justificación de cada elección:

- Python 3.12 — lenguaje del backend; sintaxis clara y curva de aprendizaje corta, ideal para un desarrollo ágil e iterativo.
- FastAPI — framework del API REST. Se eligió porque genera documentación interactiva (Swagger UI) de forma automática, sin escribir una sola línea de frontend, lo cual acelera tanto el desarrollo como la demostración del sistema.
- SQLAlchemy 2.0 + SQLite — ORM y motor de base de datos embebido. No requiere instalar ni configurar un servidor de base de datos aparte, por lo que el proyecto corre en cualquier máquina sin pasos adicionales.
- Pydantic v2 — validación de datos de entrada y salida, integrado nativamente con FastAPI.
- pytest y pytest-html — framework de pruebas automatizadas y generación de reportes de evidencia (justificación ampliada en la sección 3.3).
- Git y GitHub — control de versiones y repositorio remoto del código fuente.
- Jira (Atlassian) — gestión del backlog, las épicas y las historias de usuario.

1.3 Objetivo del proyecto

Desarrollar un sistema que centralice y automatice el agendamiento de citas entre clientes y profesionales, sustituyendo procesos manuales propensos a error (agendas en papel, mensajería informal, hojas de cálculo) por una API que garantiza, mediante validaciones automáticas, que un profesional nunca quede agendado en dos citas que se crucen, que un cliente no reserve fuera del horario disponible, y que toda cita cumpla las reglas mínimas de negocio antes de confirmarse.

1.4 Alcance del proyecto

El primer Release del sistema incluye:

- Registro y consulta de clientes.
- Registro y consulta de profesionales, incluyendo su horario laboral.
- Agendamiento de citas, aplicando las seis reglas de negocio descritas en la sección 1.6.1.
- Confirmación y cancelación de citas.
- Documentación interactiva del API (Swagger UI) para probar cada funcionalidad desde el navegador.

Explícitamente fuera del alcance de este primer Release:

- Interfaz gráfica de usuario (frontend web o móvil).
- Envío de notificaciones automáticas (correo electrónico, SMS).
- Cobros, pagos o facturación.
- Soporte para múltiples sucursales.
- Autenticación y control de acceso de usuarios (login).

1.5 Cronograma del proyecto

El proyecto se ejecutó en una sesión de trabajo intensiva y continua, estructurada en las siguientes actividades:

#	Actividad	Entregable	Horario	Responsable
1	Levantamiento de requerimientos y diseño del modelo de datos	Alcance y modelo de datos definidos	17/08/2026, 6:30 p.m. – 6:45 p.m.	Brian Guzmán
2	Configuración del entorno y repositorio	Repositorio inicial en GitHub	17/08/2026, 6:45 p.m. – 7:00 p.m.	Brian Guzmán
3	Desarrollo: gestión de clientes y profesionales	Endpoints CRUD funcionando	17/08/2026, 7:00 p.m. – 7:30 p.m.	Brian Guzmán
4	Desarrollo: agendamiento y las 6 reglas de negocio	6 reglas implementadas y probadas	17/08/2026, 7:30 p.m. – 9:00 p.m.	Brian Guzmán
5	Carga de épicas e historias de usuario en Jira	Tablero Scrum completo	17/08/2026, 9:00 p.m. – 9:30 p.m.	Brian Guzmán
6	Redacción del documento del proyecto	Este documento	17/08/2026, 9:30 p.m. – 10:45 p.m.	Brian Guzmán
7	Grabación del video demostrativo	Video del incremento	17/08/2026, 10:45 p.m. – 11:10 p.m.	Brian Guzmán
8	Formato final y entrega	PDF, links y entrega oficial	17/08/2026, 11:10 p.m. – 11:59 p.m.	Brian Guzmán

1.6 Definición del primer Release

El primer Release entrega un backend completamente funcional para la gestión de clientes, profesionales y citas, con todas las reglas de negocio activas y cubiertas por pruebas automatizadas.

1.6.1 Requerimientos funcionales

ID	Descripción
RF-01	El sistema debe permitir registrar un cliente con nombre, correo y teléfono.
RF-02	El sistema debe impedir el registro de un cliente con un correo ya existente.
RF-03	El sistema debe permitir consultar y listar los clientes registrados.
RF-04	El sistema debe permitir registrar un profesional junto con su horario laboral.
RF-05	El sistema debe permitir consultar los profesionales registrados.
RF-06	El sistema debe permitir agendar una cita para un cliente con un profesional.
RF-07	El sistema debe rechazar una cita que se solape con otra cita activa del mismo profesional.
RF-08	El sistema debe rechazar una cita que no quepa dentro del horario laboral del profesional.
RF-09	El sistema debe rechazar una cita agendada en el pasado.
RF-10	El sistema debe rechazar una duración de cita que no sea múltiplo de 30 minutos.
RF-11	El sistema debe rechazar una cita si el cliente ya tiene otra cita activa que se solape, aunque sea con otro profesional.
RF-12	El sistema debe permitir confirmar una cita en estado PENDIENTE.
RF-13	El sistema debe permitir cancelar una cita activa, exigiendo un mínimo de 24 horas de anticipación.
RF-14	El sistema debe permitir listar citas filtrando por profesional, cliente o estado.

1.6.2 Requerimientos no funcionales

ID	Descripción
RNF-01	El sistema debe exponer documentación interactiva del API (Swagger UI) sin configuración adicional.
RNF-02	El sistema debe validar todos los datos de entrada antes de persistirlos en la base de datos.
RNF-03	El sistema debe responder con códigos de estado HTTP estándar (200, 201, 404, 409, 422) según el resultado de cada operación.
RNF-04	El sistema debe contar con una suite de pruebas automatizadas que cubra cada regla de negocio con al menos un caso de aceptación y uno de rechazo.
RNF-05	El sistema debe poder ejecutarse localmente con un solo comando, sin depender de servicios externos.

2. Metodología Scrum

2.1 Definición de tareas a ejecutar

El trabajo se dividió en las siguientes tareas técnicas, agrupadas por componente:

- Diseñar el modelo de datos (Cliente, Profesional, Cita) y su persistencia con SQLAlchemy.
- Implementar el CRUD de clientes con validación de correo único.
- Implementar el CRUD de profesionales con horario laboral.
- Implementar el endpoint de agendamiento de citas.
- Implementar, una por una, las seis reglas de negocio del agendamiento.
- Implementar los endpoints de confirmar y cancelar una cita.
- Escribir las pruebas automatizadas de cada funcionalidad y cada regla.
- Configurar el reporte de evidencia de ejecución de pruebas (pytest-html).
- Documentar el proyecto y preparar la demostración en video.

2.2 Definición del equipo de trabajo

Este es un proyecto individual. Por su naturaleza académica y su alcance acotado, un solo integrante asume los tres roles de Scrum de forma secuencial según la actividad que se esté realizando, práctica común en equipos pequeños:

Rol	Responsable	Responsabilidades	Habilidad clave
Product Owner	Brian Guzmán Manon	Define el objetivo del producto, prioriza el backlog y acepta las historias completadas.	Entendimiento del dominio del negocio (reglas de agendamiento).
Scrum Master	Brian Guzmán Manon	Facilita el proceso, organiza el trabajo por sprints y vela por el cumplimiento del plan.	Organización y gestión del tiempo.
Development Team	Brian Guzmán Manon	Implementa el backend, escribe las pruebas automatizadas y documenta el sistema.	Python, FastAPI, pruebas automatizadas.

2.3 Herramientas y definición de épicas

La gestión de historias de usuario y el tablero Scrum se llevan en Jira (Atlassian), en un proyecto tipo Scrum con backlog, sprints y tablero Kanban. El backlog se organiza en cuatro épicas:

- Gestión de Clientes — registro y consulta de clientes.
- Gestión de Profesionales — registro y consulta de profesionales y su horario.
- Agendamiento — creación de citas y las cinco reglas de negocio que se validan al agendar.
- Ciclo de Vida de la Cita — confirmación y cancelación de citas.

2.4 Ceremonias de Scrum

Ceremonia	Fecha y hora	Duración	Participantes	Objetivo
Sprint Planning	17/08/2026, 6:30 p.m.	15 min	Brian Guzmán	Definir el alcance del primer Release y las tareas del sprint.
Daily Stand-up	17/08/2026, cada bloque de trabajo	15 min	Brian Guzmán	Revisar avance y bloqueos entre bloques de desarrollo.
Sprint Review	17/08/2026, 10:45 p.m.	20 min	Brian Guzmán	Demostrar el incremento del sistema (grabación del video).
Sprint Retrospective	17/08/2026, 11:00 p.m.	10 min	Brian Guzmán	Identificar qué mejorar (documentado en la sección de Conclusiones).

2.5 Historias de usuario

Se definieron diez historias de usuario, agrupadas en las cuatro épicas del backlog y cargadas en el tablero Jira con sus criterios de aceptación y puntos de historia. Los puntos se estimaron con escala de Fibonacci (1, 2, 3, 5, 8), usando como referencia la historia más simple (HU-02, consultar clientes, valor 1).

ID	Épica	Historia	Pts.
HU-01	Gestión de Clientes	Como recepcionista quiero registrar un cliente para poder agendarle citas.	2
HU-02	Gestión de Clientes	Como recepcionista quiero consultar y listar clientes para elegir a quién agendarle una cita.	1
HU-03	Gestión de Profesionales	Como administrador quiero registrar un profesional con su horario para que el sistema sepa cuándo puede recibir citas.	2
HU-04	Gestión de Profesionales	Como recepcionista quiero consultar los profesionales y su horario para saber a quién asignar una cita.	1
HU-05	Agendamiento	Como recepcionista quiero agendar una cita para un cliente con un profesional para reservar el servicio.	3
HU-06	Agendamiento	Como administrador quiero que el sistema rechace citas que se crucen con otra del mismo profesional para no comprometerlo en dos lugares a la vez.	5
HU-07	Agendamiento	Como administrador quiero que el sistema rechace citas fuera del horario del profesional para no agendar fuera de su disponibilidad real.	3
HU-08	Agendamiento	Como administrador quiero que el sistema rechace citas en el pasado o con duración inválida para mantener la agenda consistente.	2
HU-09	Agendamiento	Como administrador quiero que el sistema rechace una cita si el cliente ya tiene otra activa que se solape, aunque sea con otro profesional.	3
HU-10	Ciclo de Vida de la Cita	Como recepcionista quiero confirmar una cita pendiente o cancelarla con anticipación suficiente para gestionar cambios de agenda.	3

Nota sobre HU-06 (5 puntos, la de mayor esfuerzo estimado): durante su implementación, la prueba automatizada detectó un error real en la lógica de comparación de rangos de fechas, que fue corregido antes de continuar. Ese hallazgo confirma la estimación de 5 puntos frente a historias similares de 3, y se documenta con más detalle en la sección 3.7.

3. Plan de Pruebas

3.1 Requerimientos relacionados con las historias de usuario

Trazabilidad entre requerimientos funcionales, historias de usuario y los casos de prueba que los verifican:

Requerimiento	Historia	Casos de prueba
RF-01, RF-02, RF-03	HU-01, HU-02	CP-002 a CP-006
RF-04, RF-05	HU-03, HU-04	CP-007 a CP-009
RF-06	HU-05	CP-010 a CP-013
RF-07	HU-06	CP-019, CP-020, CP-026
RF-08	HU-07	CP-021, CP-022
RF-09	HU-08	CP-023
RF-10	HU-08	CP-024
RF-11	HU-09	CP-025
RF-12, RF-13, RF-14	HU-10	CP-014 a CP-018, CP-027, CP-028
RNF-01 a RNF-05	Todas	Verificado de forma transversal (ver 3.6 y 3.7)

3.2 Criterios de aceptación y rechazo

Un caso de prueba se considera Aprobado cuando la respuesta del API cumple dos condiciones a la vez: el código de estado HTTP coincide con el esperado, y el efecto en la base de datos es el esperado (por ejemplo, que la cita quede en el estado correcto, o que no se haya creado ningún registro cuando la operación debía rechazarse).

Un caso de prueba se considera Fallido cuando el código de estado HTTP no coincide con el esperado, o cuando hay un efecto secundario no deseado (por ejemplo, que una cita inválida termine guardada en la base de datos).

3.3 Herramientas de pruebas justificadas

Herramienta	Justificación
pytest	Estándar de facto para pruebas en Python. Su sintaxis basada en assert simples y su sistema de fixtures reducen el código repetitivo frente a unittest, que exige clases y métodos assertEquals.
pytest-html	Genera un reporte HTML autocontenido de la ejecución, que sirve como evidencia sin depender de un servidor de integración continua.
TestClient de FastAPI (basado en httpx)	Permite probar cada endpoint sin levantar un servidor real, lo que hace las pruebas rápidas y deterministas.
SQLite en memoria por prueba	Cada prueba recibe una base de datos limpia y aislada, evitando que el resultado de una prueba dependa del orden de ejecución o contamine a otra.

3.4 Cronograma de ejecución de pruebas

Tipo	Alcance	Casos	Momento de ejecución	Responsable
Automatizada	Endpoints CRUD (clientes y profesionales)	CP-002 a CP-009	Durante el desarrollo de cada endpoint	Brian Guzmán
Automatizada	Flujo de citas (crear, confirmar, cancelar, listar)	CP-010 a CP-018	Durante el desarrollo del flujo de citas	Brian Guzmán
Automatizada	Las seis reglas de negocio	CP-019 a CP-028	Una regla a la vez, en ciclo rojo-verde	Brian Guzmán
Manual — exploratoria	Flujo completo desde Swagger UI	—	Antes de grabar el video demostrativo	Brian Guzmán
Manual — regresión	Suite completa (pytest) más revisión de enlaces	28 casos	Justo antes de la entrega	Brian Guzmán

3.5 Plantillas para casos de prueba y equipos de pruebas

Formato utilizado para documentar cada caso de prueba:

Campo	Contenido
ID del caso	CP-000
Requerimiento asociado	RF-00 / RNF-00
Historia de usuario	HU-00
Precondiciones	Estado que debe existir antes de ejecutar la prueba
Pasos	Secuencia de acciones sobre el API
Resultado esperado	Código HTTP y efecto esperado en la base de datos
Tipo	Automatizada / Manual
Estado	Aprobada / Fallida

Listado de los 28 casos de prueba automatizados que componen la suite:

ID	Título	Tipo
CP-001	El servicio responde y reporta estado operativo	Automatizada
CP-002	Crear cliente con datos válidos	Automatizada
CP-003	Rechazar cliente con correo duplicado	Automatizada
CP-004	Rechazar correo con formato inválido	Automatizada
CP-005	Consultar cliente inexistente devuelve 404	Automatizada
CP-006	Listado de clientes incluye el creado	Automatizada
CP-007	Crear profesional con horario válido	Automatizada
CP-008	Rechazar horario invertido (fin antes que inicio)	Automatizada
CP-009	Consultar profesional inexistente devuelve 404	Automatizada
CP-010	Crear cita con datos válidos	Automatizada
CP-011	Rechazar cita con cliente inexistente	Automatizada
CP-012	Rechazar cita con profesional inexistente	Automatizada
CP-013	Rechazar duración fuera del rango permitido (422)	Automatizada

ID	Título	Tipo
CP-014	Confirmar una cita pendiente	Automatizada
CP-015	Rechazar confirmar una cita dos veces	Automatizada
CP-016	Cancelar una cita activa	Automatizada
CP-017	Rechazar cancelar una cita dos veces	Automatizada
CP-018	Listado de citas filtrado por profesional	Automatizada
CP-019	Rechazar solapamiento de citas del mismo profesional	Automatizada
CP-020	Permitir citas pegadas (sin solapar)	Automatizada
CP-021	Rechazar cita antes del horario de apertura	Automatizada
CP-022	Rechazar cita que termina después del cierre	Automatizada
CP-023	Rechazar cita agendada en el pasado	Automatizada
CP-024	Rechazar duración que no es múltiplo de 30 minutos	Automatizada
CP-025	Rechazar cliente con dos citas simultáneas	Automatizada
CP-026	Una cita cancelada libera el horario	Automatizada
CP-027	Rechazar cancelación sin 24h de anticipación	Automatizada
CP-028	Permitir cancelación con anticipación suficiente	Automatizada

Equipo de pruebas: al ser un proyecto individual, Brian Guzmán Manon es responsable del diseño, la implementación y la ejecución de los 28 casos de prueba.

3.6 Plan de automatización de pruebas

Las seis reglas de negocio se implementaron siguiendo un ciclo de prueba-antes-que-código: por cada regla, primero se escribió la prueba que describe el comportamiento esperado, se verificó que fallara (porque la regla aún no existía), se implementó la regla, y finalmente se confirmó que la prueba pasara. Este orden se documenta con detalle porque durante la Regla 1 (solapamiento de citas) la prueba detectó un error real en la comparación de rangos de fechas, corregido antes de seguir con las siguientes reglas.

Cada prueba usa una base de datos SQLite en memoria, creada y destruida por separado en cada caso (mediante fixtures de pytest), lo que garantiza pruebas deterministas e independientes del orden de ejecución. El comando `pytest` ejecuta la suite completa y genera automáticamente el reporte de evidencia en formato HTML.

3.7 Ejecución y demostración

La suite completa de 28 pruebas automatizadas se ejecuta con el comando `pytest` y produce el reporte `reporte-pruebas.html`, incluido como evidencia junto con este documento. El resultado de la última ejecución fue: 28 aprobadas, 0 fallidas.

Como evidencia adicional del proceso real de desarrollo (y no solo del resultado final): al implementar la Regla 1, la prueba automatizada de solapamiento de citas falló en el primer intento porque la función de comparación de rangos tenía una condición invertida. El error se corrigió y la prueba pasó a estar en verde. Ese mismo patrón se repitió al implementar la Regla 6 (cancelación con 24h de anticipación), donde dos pruebas que ya estaban en verde empezaron a fallar porque la fecha de prueba usada hasta ese momento

(«mañana a las 9:00») dejó de cumplir el mínimo de 24 horas dependiendo de la hora del día en que se ejecutara la suite; se corrigió creando una fecha de prueba distinta con holgura garantizada.

4. Demostración y Entregables

4.1 Video

El video demostrativo muestra el incremento del sistema construido en este primer Release: creación de un cliente y un profesional, agendamiento de una cita válida, un intento de agendamiento que viola una regla de negocio (y su rechazo), confirmación y cancelación de una cita, todo ejecutado desde la documentación interactiva del API (Swagger UI). El video cierra con la ejecución de la suite de pruebas automatizadas desde la terminal, mostrando el resultado en verde.

4.2 Links funcionales

Conforme a las reglas de entrega de la plataforma, los enlaces al repositorio de código, al tablero de gestión (Jira) y al video demostrativo se entregan mediante la opción “Texto en línea” de la plataforma de la tarea, y no se incluyen dentro de este documento PDF.

- Repositorio del código fuente (incluye el código de las pruebas automatizadas).
- Tablero de gestión en Jira, con las épicas y las diez historias de usuario.
- Video demostrativo del incremento del sistema.

Conclusiones

Lo más difícil para mí durante el proyecto fue asegurar que las pruebas fueran realmente confiables y que no dependieran de condiciones externas, especialmente de la fecha y hora en que se ejecutaban. La implementación de nuevas reglas también me obligó a revisar pruebas que anteriormente funcionaban, lo que me enseñó que una suite de pruebas debe mantenerse y evolucionar junto con el sistema. Si tuviera más tiempo, dedicaría una parte mayor del desarrollo inicial a diseñar casos límite y escenarios de prueba antes de implementar cada regla, para detectar antes posibles supuestos o inconsistencias. Para un segundo entregable, agregaría funcionalidades que aporten mayor valor al usuario, como autenticación y diferentes roles de usuario, un sistema de notificaciones y recordatorios de citas, y una interfaz que permita consultar y gestionar las citas de forma sencilla. También incorporaría más validaciones y pruebas de integración para asegurar que las nuevas funcionalidades funcionen correctamente junto con las seis reglas de negocio existentes.

Bibliografía

FastAPI. (2024). FastAPI documentation. <https://fastapi.tiangolo.com/>

SQLAlchemy. (2024). SQLAlchemy 2.0 documentation. <https://docs.sqlalchemy.org/>

Pydantic. (2024). Pydantic documentation. <https://docs.pydantic.dev/>

pytest. (2024). pytest documentation. <https://docs.pytest.org/>

Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum. Scrum.org.

Atlassian. (2024). Jira Software documentation. <https://www.atlassian.com/software/jira>